

The Lens, Part 3: उत्तराचं जग

August 30, 2026

THE LENS

Book 7, Part 3: उताराचं जग

मागच्या भागाचा शेवट: थेंबूला चूक (loss) मोजता येते, पण प्रश्न उरला: ती कमी करायला weights कुठल्या दिशेने, किती हलवायचे? हा भाग त्याचं उत्तर: आणि तेच पूर्ण यंत्र-शिक्षणाचं हृदय आहे.

Derivative (बदलाचा दर), gradient (उतार), chain rule (चूक मागे वाटणं), आणि सर्वात महत्वाचं: थेंबूवर पूर्ण **backpropagation**, हाताने, **खऱ्या आकड्यांनी**. हा भाग संपला की “यंत्र शिकतं” हे वाक्य तुमच्यासाठी कायमचं उघडं होईल.

सूत्र (पुन्हा)

गणित म्हणजे आकडेमोड नव्हे; न दिसणारं दाखवणारा चष्मा.
जे मोजता येतं, ते सुधारता येतं.
Part 3 चा भर: चूक कुठे-किती हे मोजता आलं की सुधारणा आपोआप होते. हेच शिकणं.

Part 2 ची उजळणी: 3 प्रश्न (बोलून; उत्तरं खाली)

1. Loss (cross-entropy) नक्की काय मोजतं?

खऱ्या घटनेला यंत्राने किती probability दिली (जास्त=कमी loss); “अशक्य म्हणणं” = स्फोटक शिक्षा. (2.5)

2. Softmax काय करतं?

कुठलेही गुण -> probability-वाटणी (स्फोट + भागा), बेरीज 1. (2.6)

3. Temperature ची कळ?

distribution तीव्र (ठाम) की सपाट (धीट) = entropy ची कळ. (2.6)

या भागात काय

Derivative (एका आकड्याचा दुसऱ्यावर परिणाम), gradient (सगळ्या weights ची उतार-दिशा), chain rule (चूक थरांमागे वाटणं), थेंबूवर पूर्ण एक शिकवणी-पाऊल हाताने, learning rate (पावलाचा आकार), आणि “हे का चालतं” चं प्रामाणिक गूढ. सगळं आकड्यांनी.

विभाग 3: उताराचं जग

Book 2 चं शिकणं आठवा: अंदाज -> चूक -> उतार -> केसभर पाऊल -> पुन्हा (2.6). त्यातला “उतार” हा या विभागाचा गाभा आहे. यंत्राकडे लाखो-अब्जावधी weights; loss कमी करायला प्रत्येकाला **किती, कुठे** ढकलायचं हे “उतार” ठरवतो. आणि आश्चर्य हे की हा उतार काढणारं गणित (derivative, gradient, chain rule) आश्चर्यकारक साधं आहे: थेंबूवर हाताने करता येईल इतकं. **हे तुमच्या शिक्षणात कुठे येईल:** “training”, “fine-tuning”, “gradient descent”, “backprop”: हे सगळे शब्द इथून जन्मतात; हा विभाग समजला की “यंत्राला शिकवणं” ही जादू न राहता अंकगणित होतं.

Chapter 3.1 [SPINE]: Derivative: बदलाचा दर

उतार समजायला आधी एक साधी कल्पना: **derivative** (बदलाचा दर). आणि ती इतकी रोजची आहे की तुम्ही ती वापरता, फक्त नाव माहीत नाही.

Derivative म्हणजे काय. एका आकड्याला थोडं ढकललं, तर दुसरा आकडा **किती** बदलतो? गाडीचा वेग म्हणजे derivative: “वेळ थोडा पुढे गेला, तर अंतर किती वाढलं.” थेंबूवर: “weight थोडं वाढवलं, तर loss किती बदलली?” हाच प्रश्न, आणि त्याचं उत्तर = derivative. एका ओळीत: **derivative = परिणामाचा दर = “इथे थोडं हललं तर तिथे किती हलतं.”**

आकड्यांनी बघू (थेंबूचं सोपं रूप). समजा थेंबूचा loss हा एका weight w वर असा अवलंबून आहे: $\text{loss} = (w - 3)^2$. म्हणजे $w = 3$ ला loss शून्य (सर्वोत्तम), आणि w जसजसा 3 पासून दूर, तसतशी loss वाढते (एक खोरं, तळाशी $w=3$). आता derivative विचारतो: “ $w = 5$ ला उभं राहून थोडं हललं तर loss कुठे जाईल?”

$$\begin{aligned} w = 5: \quad \text{loss} &= (5-3)^2 = 4 \\ w = 5.1: \text{loss} &= (5.1-3)^2 = 2.1^2 = 4.41 \quad (\text{वाढली!}) \\ w = 4.9: \text{loss} &= (4.9-3)^2 = 1.9^2 = 3.61 \quad (\text{घटली!}) \end{aligned}$$

म्हणजे $w = 5$ ला: w **वाढवलं** तर loss वाढते; w **घटवलं** तर loss घटते. Derivative हा दर आणि दिशा दोन्ही सांगतो: इथे तो **धन** (positive) आहे (“ w वाढला की loss वाढते”), आणि आकार सांगतो किती वेगात. आणि इथे शिकण्याची गुरुकिल्ली: **loss कमी करायची तर derivative च्या उलट दिशेला जा.** derivative धन (w वाढला की loss वाढते) $\rightarrow$ म्हणून w **कमी** करा (उताराच्या खाली, $w=3$ कडे). हेच “उतार उतरणं.”

डोंगराची उपमा (Book 2 चा धुका). $\text{loss} =$ एक डोंगराळ भूभाग, उंची = चूक. तुम्ही एका जागी ($w = 5$) उभे, तळ (कमीत कमी loss) शोधायचाय, पण धुक्यामुळे पूर्ण नकाशा दिसत नाही. Derivative म्हणजे: **तुमच्या पायाखालचा उतार:** “इथे जमीन कुठल्या दिशेला झुकलीये, किती तीव्र.” उतार कळला की एक पाऊल **खाली** टाका (loss कमी होईल त्या दिशेला). पुन्हा उतार बघा, पुन्हा पाऊल. हळूहळू तळाकडे. Derivative = पायाखालच्या उताराचं मोजमाप: पूर्ण नकाशा नाही, पण पुढचं एक पाऊल पुरेसं.

इथे लोक काय चुकीचं समजतात

”Derivative म्हणजे किचकट calculus, सूत्रं.” मूळ कल्पना साधी: “इथे थोडं हललं तर तिथे किती हलतं.” तुम्ही ती रोज वापरता (वेग = अंतराचा derivative, प्रवेग = वेगाचा). AI मध्ये लागणारं derivative फक्त काही मूठभर नियमांचं (कोणतं फार महत्वाचं नाही; कल्पना महत्वाची). आणि दुसरा गैरसमज: “derivative म्हणजे उत्तर.” नाही: derivative म्हणजे दिशा (पुढचं पाऊल कुठे), उत्तर नाही: उत्तर मिळतं अनेक पावलं टाकून (3.4).

नकाशावर

(दर chapter ला एक ओळ.)

3.1 derivative = बदलाचा दर (“इथे हललं तर तिथे किती”);
पायाखालचा उतार; loss कमी करायला उलट दिशेला जा

सूत्रावर: derivative हा चष्मा “सुधारायचं तर कुठे सरकावं” हे **मोजून** दाखवतो: अंदाजाने नाही, दराने. जे मोजता येतं (उतार) त्या दिशेने सुधारता येतं (खाली पाऊल): हेच शिकण्याचं यांत्रिक हृदय.

स्वतः बघा (5 मिनिटं)

डोंगरउतारावर चेंडू सोडला तर तो **सर्वात तीव्र उताराच्या** दिशेने घरंगळतो, तळाकडे. का? कारण गुरुत्व त्याला “derivative च्या उलट” ढकलतं. यंत्राचं शिकणं तेच: loss-डोंगरावर weight-चेंडू उताराने खाली. निसर्ग हेच गणित वापरतो; आपण फक्त त्याला आकड्यांत मांडलं.

विचार करा

थेंबूला एकच weight असेल तर उतार सोपा (एका दिशेला खाली). पण थेंबूला **दोन** weights आहेत (आर्द्रता आणि ढगांचे), आणि खऱ्या यंत्राला अब्जावधी. “एका weight चा उतार कळला; पण लाखो weights एकाच वेळी, प्रत्येकाचा loss वर वेगळा परिणाम: सगळ्यांचे उतार एकत्र कसे काढायचे, आणि कुठल्या दिशेला सगळ्यांनी मिळून जायचं?”

सगळ्या weights चे उतार एकत्र म्हणजे एक “उतार-बाण” (Part 1 चा vector!): तो सगळ्यात तीव्र-उताराची दिशा दाखवतो. त्याचं नाव gradient. पुढचा chapter.

Chapter 3.2 [SPINE]: Gradient: धुक्यातला डोंगर उतरण

थेंबूचा प्रश्न: लाखो weights चे उतार एकत्र कसे? उत्तर Part 1 शी जोडतं: सगळ्या weights चे derivatives मिळून एक **बाण** (vector) होतो, आणि त्याचं नाव **gradient**. हा विभागाचा मध्यवर्ती शब्द आहे.

Gradient म्हणजे काय. एका weight चा derivative = त्या एका दिशेतला उतार (3.1). पण अनेक weights म्हणजे अनेक दिशा (थेंबूला 2: आर्द्रता-अक्ष, ढग-अक्ष). प्रत्येक अक्षाचा वेगळा उतार काढा, त्यांची यादी बनवा: **तोच gradient बाण**. आणि त्याचा सुंदर गुणधर्म: **gradient बाण नेहमी सर्वात तीव्र चढाच्या दिशेला दाखवतो**. म्हणजे “इथून सर्वात वेगात वर (loss वाढेल) कुठे?” मग loss कमी करायला? **gradient च्या बरोबर उलट** जा: सर्वात तीव्र **उताराची** दिशा. याला **gradient descent** (उतार-उतरण) म्हणतात: पूर्ण यंत्र-शिक्षणाचं इंजिन.

धुक्यातल्या डोंगराची पूर्ण उपमा (Book 2, 2.6 आता नेमकी). तुम्ही धुक्यात डोंगरावर, तळ शोधायचाय (कमीत कमी loss). पूर्ण नकाशा दिसत नाही. काय करता? (1) पायाखाली चाचपून बघता: **कुठली दिशा सर्वात तीव्र खाली** (gradient चं उलट). (2) त्या दिशेला **एक पाऊल** (छोटं, कारण धुकं). (3) नव्या जागी पुन्हा चाचपा, पुन्हा पाऊल. हळूहळू तळाकडे. हेच gradient descent: **उतार बघ -> उलट दिशेला पाऊल -> पुन्हा**, हजारो- लाखो वेळा. यंत्र “शिकतं” म्हणजे नक्की हेच करतं: loss-डोंगरावर weights-चेंडू धुक्यात, पावलापावलाने तळाकडे.

थेंबूवर आकड्यांनी (दोन weights). थेंबूचा loss दोन weights w_1 , w_2 वर. एका जागी gradient काढला, म्हणा तो बाण (+4, -2) निघाला. अर्थ: “ w_1 वाढवला तर loss जोरात वाढेल (+4); w_2 वाढवला तर loss घटेल (-2).” loss कमी करायला **उलट** जा: w_1 **कमी** करा, w_2 **वाढवा**, आणि प्रमाणात (4 चा उतार मोठा म्हणून w_1 जास्त हलवा). एक पाऊल:

```
gradient = (+4, -2)    (सर्वात तीव्र चढाची दिशा)
उलट दिशा = (-4, +2)   (सर्वात तीव्र उताराची: इकडे जा)
w1: 5 -> 5 - (थोडं)x4  (कमी करा, जास्त)
w2: 3 -> 3 + (थोडं)x2  (वाढवा, कमी)
```

नवे weights loss किंचित कमी करतात. पुन्हा gradient, पुन्हा पाऊल. हेच शिकणं, फक्त 2 ऐवजी अब्जावधी weights आणि ट्रिलियन पावलं.

आणि इथे GPU का लागतो ते नेमकं दिसतं (Book 2 चा कारखाना): अब्जावधी weights चा gradient एका पावलात काढणं म्हणजे अब्जावधी derivatives, अब्जावधी गुणा-बेरीज (Part 1 चे dot products, matrices). आणि हे ट्रिलियन पावलं. म्हणून हजारो GPU, महिनोन्महिने (Book 4, 1.1 चं training-bill: आता कळलं कशाचं). “training” म्हणजे: प्रचंड gradient descent, धुक्यातल्या अवाढव्य डोंगरावर.

इथे लोक काय चुकीचं समजतात

“Gradient descent यंत्राला ‘सर्वोत्तम उत्तर’ शोधून देतो.” नाही: तो फक्त **जवळचा खोलगट भाग** शोधतो (local minimum), जगातला सर्वात खोल नाही. धुक्यात तुम्ही एका दरीत उतराल, पण शेजारी अजून खोल दरी असू शकते: तुम्हाला दिसत नाही. आश्चर्य हे की अब्जावधी-मितींच्या डोंगरावर हे बहुधा **पुरेसं चांगलं** निघतं (का, ते 3.6 चं गूढ). पण “सर्वोत्तम सापडतं” ही चूक: gradient descent “चांगलं-पुरेसं” शोधतो, “परिपूर्ण” नाही.

नकाशावर

- 3.1 derivative = एका दिशेतला उतार
- 3.2 gradient = सगळ्या weights चे उतार एकत्र (बाण); सर्वात तीव्र चढाची दिशा -> उलट जा (gradient descent); धुक्यात पावलापावलाने तळाकडे = शिकणं

सूत्रावर: gradient हा चष्मा “अब्जावधी weights एकत्र कसे सुधारायचे” या अशक्य वाटणाऱ्या प्रश्नाला एका बाणात आणतो: सर्वात तीव्र उताराची दिशा. जे मोजता येतं (gradient) त्या दिशेने सगळे weights एकदम सुधारतात: हेच शिकण्याचं इंजिन.

स्वतः बघा (5 मिनिटं)

डोळे मिटून, एका उतारावर उभं असल्याची कल्पना करा. पायाखाली जाणवतं: डावा पाय थोडा खाली, उजवा थोडा वर -> म्हणजे उतार डावीकडे-खाली. तुम्ही सहज डावीकडे पाऊल टाकाल (तळाकडे). तुमचा पाय हा gradient-sensor आहे; तुमचं पाऊल gradient descent. यंत्र हेच करतं, फक्त अब्जावधी “पायांनी” (weights) एकाच वेळी.

विचार करा

थेंबूचं यंत्र आता एका थराचं समजलं. पण खरं यंत्र **अनेक थरांचं** (Part 1, 1.4: साखळी). शेवटच्या थरात loss दिसते; पण **पहिल्या** थरातल्या weight ने त्या loss वर किती परिणाम केला हे कसं काढायचं? “पहिला थर -> दुसरा -> ... -> शेवटचा -> loss: इतक्या लांब साखळीत, सुरुवातीच्या एका weight चा शेवटच्या loss वरचा उतार कसा मोजायचा?”

हाच deep learning चा सर्वात हुशार भाग, आणि तो एका साध्या गुणाकार-नियमाने सुटतो: चूक थरामागून थर, मागे वाटत जाणं. त्याचं नाव chain rule / backpropagation: पुढचा chapter, विभागाचा गाभा.

Chapter 3.3 [SPINE]: Chain Rule: चूक मागे वाटणं

थेंबूचा प्रश्न: लांब साखळीत सुरुवातीच्या weight चा शेवटच्या loss वरचा उतार कसा? उत्तर deep learning चा सर्वात हुशार, सर्वात सुंदर भाग आहे: **chain rule** (साखळी-नियम), ज्याला कृतीत **backpropagation** (चूक मागे-वाटणं) म्हणतात.

कल्पना: परिणामांची साखळी. एक साखळी: पाऊस कमी -> धरणात पाणी कमी -> वीज कमी -> कारखाना बंद -> मजूर बेकार. आता प्रश्न: “पाऊस 10% कमी झाला तर मजुरीवर किती परिणाम?” उत्तर: **प्रत्येक दुव्याचा दर गुणा.** पाऊस->पाणी चा दर x पाणी-> वीज चा दर x वीज->कारखाना x कारखाना->मजुरी. दरांचा गुणाकार = टोकापासून टोकाचा परिणाम. **हाच chain rule: साखळीतल्या प्रत्येक पायरीचा दर (derivative) गुणा, म्हणजे सुरुवातीचा शेवटावरचा परिणाम मिळतो.**

यंत्रात: loss मागे वाटणं. थेंबूचं यंत्र: पहिला थर -> दुसरा -> शेवटचा -> loss. पहिल्या थरातल्या weight चा loss वरचा उतार हवाय. Chain rule: **शेवटापासून सुरुवातीकडे, प्रत्येक थराचा दर गुणत मागे या.** म्हणून याला “back-propagation”: loss ची “जबाबदारी” शेवटच्या थरापासून मागे, प्रत्येक थराला त्याच्या वाट्याची, वाटत जाणं. जणू loss म्हणते “ही चूक झाली; प्रत्येक थराचा यात किती हात?": आणि उत्तर मागे-मागे, गुणाकाराने, प्रत्येक weight पर्यंत पोहोचतं.

दोन पावलं (हेच पूर्ण training चं यंत्र):

1. पुढे (forward): माहिती आत -> थर-थर -> अंदाज -> loss.
(थेंबू: हवा -> गुण -> probability -> चूक)
2. मागे (backward): loss -> chain rule ने प्रत्येक थराचा दर गुणत मागे -> प्रत्येक weight चा उतार (gradient).
मग gradient descent ने सगळे weights एक पाऊल सुधार (3.2).

पुढे-मागे, पुढे-मागे: ट्रिलियन वेळा. हेच “यंत्र शिकतं.” **Forward ने अंदाज, backward ने दोष-वाटप, मग सुधारणा.** आणि सौंदर्य: कितीही खोल यंत्र (शंभर थर) असो, chain rule प्रत्येक weight पर्यंत नेमका उतार पोहोचवतो: फक्त दर गुणत मागे या. Deep learning शक्य झालं कारण हा एक साधा गुणाकार-नियम अब्जावधी weights ना एका पावलात सुधारू देतो.

एक सूक्ष्म धोका (कळून घ्या, नाव पुरे): दर गुणत-गुणत मागे जाताना, जर दर सतत **छोटे** ($0.1 \times 0.1 \times 0.1 \dots$) असतील तर गुणाकार जवळजवळ **शून्य** होतो: सुरुवातीच्या थरांपर्यंत उतार पोहोचतच नाही (ते शिकतच नाहीत): याला “vanishing gradient” म्हणतात. उलट दर मोठे तर गुणाकार **स्फोटक** (exploding). खूप-खोल यंत्रं शिकवायला हे मोठे अडथळे होते; आणि transformer ची काही रचना

(residual connections वगैरे) नेमकी हे टाळायला: म्हणजे Book 2 चा transformer अंशतः या गणिती अडचणीवरचा उपाय आहे. (खोलात जायची गरज नाही; एवढं कळलं की papers मधला हा शब्द ओळखीचा होईल.)

इथे लोक काय चुकीचं समजतात

”Backpropagation म्हणजे यंत्र ‘मागे विचार करतं’ किंवा ‘आठवतं’.” नाही: हे विचार-आठवण नाही, **शुद्ध गुणाकार**. loss पासून मागे, प्रत्येक थराचा दर गुणत, प्रत्येक weight चा उतार काढणं: यांत्रिक, भावनाशून्य अंकगणित. “यंत्र स्वतःच्या चुकांतून शिकतं” ही कविता; “loss चा gradient chain rule ने प्रत्येक weight ला वाटला जातो आणि weights एक पाऊल सरकतात” हे गणित. Backprop ला बुद्धी नाही, फक्त गुणाकार आहे.

नकाशावर

- 3.1 derivative = उतार 3.2 gradient = उतार-बाण, उलट जा
- 3.3 chain rule/backprop: प्रत्येक थराचा दर गुणत loss मागे वाटणं; forward (अंदाज) + backward (दोष-वाटप) = शिकणं; vanishing/exploding = खोलीचा अडथळा

सूत्रावर: chain rule हा चष्मा “खोल यंत्र कसं शिकतं” या गूढाला एका गुणाकार-नियमात आणतो: चूक मागे वाटा, प्रत्येक weight ला त्याचा वाटा. जे मोजता येतं (प्रत्येकाचा दोष) ते सुधारता येतं (प्रत्येकाला एक पाऊल): deep learning चं संपूर्ण रहस्य एका गुणाकारात.

स्वतः बघा (5 मिनिटं)

एक साखळी-हिशेब करा: एका दुकानाचा नफा 10% वाढला, दुकानाचा मालक कंपनीचा 20% भागधारक, कंपनीचा नफा शहराच्या करात 5% योगदान. “दुकान-नफा वाढ शहराच्या करावर किती?”: दर गुणा (0.10 x 0.20 x 0.05 = 0.001, म्हणजे किरकोळ). तुम्ही आत्ता chain rule केलात: साखळीतले दर गुणून टोकाचा परिणाम. यंत्र loss साठी हेच करतं, फक्त लाखो दुव्यांचं.

विचार करा

थेंबूला आता सगळे तुकडे कळले: अंदाज (Part 1), चूक (Part 2), उतार, gradient, backprop. पण थेंबू म्हणतो: “तुकडे कळले, पण मला **एकदा, संपूर्ण, आकड्यांनी** करून बघायचंय: माझी खरी हवा घ्या, अंदाज काढा, चूक मोजा, आणि एक weight प्रत्यक्ष सुधारा: सुरुवातीपासून शेवटपर्यंत, काहीही न

वगळता. तरच ‘यंत्र शिकतं’ हे माझ्या हाडात मुरेल.”

हीच पुढच्या chapter ची विनंती, आणि विभागाचा कळस: थेंबूवर पूर्ण एक शिकवणी-पाऊल, हाताने, खऱ्या आकड्यांनी, एकही पायरी न वगळता. यंत्र-शिक्षण डोळ्यांसमोर घडताना बघू.

Chapter 3.4 [SPINE]: थेंबूला हाताने शिकवू: पूर्ण एक पाऊल

हा पूर्ण पुस्तकातला सर्वात महत्वाचा chapter. इथे आपण थेंबूला **प्रत्यक्ष, हाताने, एका शिकवणी-पावलातून** नेऊ: एकही पायरी न वगळता, खऱ्या आकड्यांनी. हे एकदा डोळ्यांनी घडताना बघितलं की “यंत्र शिकतं” हे वाक्य कायमचं उघडं होईल. कागद-पेन घ्या; सोबत करा.

थेंबूची रचना (सर्वात सोपी). थेंबू एका ओळीचं यंत्र:

$$\text{अंदाज } p = w_1 \times \text{आर्द्रता} + w_2 \times \text{ढग} + b$$

w_1, w_2 = आर्द्रता आणि ढगांना दिलेली महत्त्वं (weights); b = एक स्थिर सरकवण (bias). आजची माहिती: आर्द्रता = 0.8, ढग = 0.6. सुरुवातीचे (कच्चे, न-शिकलेले) weights: $w_1 = 1.0, w_2 = 1.0, b = 0$. खरं घडलं: **पूर्ण पाऊस, म्हणजे लक्ष्य $y = 1$.**

पायरी 1: पुढे (forward): अंदाज काढा.

$$\begin{aligned} p &= 1.0 \times 0.8 + 1.0 \times 0.6 + 0 \\ &= 0.8 + 0.6 \\ &= 1.4 \end{aligned}$$

थेंबूचा अंदाज 1.4. पण खरं लक्ष्य 1. म्हणजे थेंबू **अति-आत्मविश्वासाने** जास्त बोलला (1 हवं होतं, 1.4 दिलं).

पायरी 2: चूक (loss) मोजा. सोप्या $\text{loss} = (\text{अंदाज} - \text{खरं})^2$:

$$\text{loss} = (p - y)^2 = (1.4 - 1)^2 = 0.4^2 = 0.16$$

चूक 0.16. आता ती कमी करायची.

पायरी 3: मागे (backward): प्रत्येक weight चा उतार (chain rule, 3.3). प्रश्न: प्रत्येक weight थोडा बदलला तर loss किती बदलेल?

आधी loss चा अंदाजावरचा दर:

$$dLoss/dp = 2 \times (p - y) = 2 \times (1.4 - 1) = 2 \times 0.4 = 0.8$$

आता अंदाजाचा प्रत्येक weight वरचा दर ($p = w_1 \times 0.8 + w_2 \times 0.6 + b$):

$$dp/dw_1 = \text{आर्द्रता} = 0.8$$

$$dp/dw_2 = \text{ढग} = 0.6$$

$$dp/db = 1$$

chain rule ने गुणा (दर मागे वाटा):

$$w_1 \text{ चा उतार} = 0.8 \times 0.8 = 0.64$$

$$w_2 \text{ चा उतार} = 0.8 \times 0.6 = 0.48$$

$$b \text{ चा उतार} = 0.8 \times 1 = 0.80$$

तिन्ही उतार **धन** (positive): अर्थ “हा weight वाढवला तर loss वाढेल.” म्हणून तिन्ही weights **कमी** करायचे (उलट दिशा: 3.2). आणि लक्षात घ्या: w_1 चा उतार (0.64) w_2 पेक्षा (0.48) मोठा, कारण आर्द्रता (0.8) ढगांपेक्षा (0.6) जास्त होती: **जो input मोठा, त्याचा weight जास्त जबाबदार, म्हणून जास्त सुधारणा**. गणित आपोआप न्याय करतं.

पायरी 4: सुधारा (gradient descent). प्रत्येक weight मधून “पावलाचा-आकार (learning rate) x उतार” वजा करा. पाऊल-आकार 0.1 घेऊ:

$$w_1: 1.0 - 0.1 \times 0.64 = 1.0 - 0.064 = 0.936$$

$$w_2: 1.0 - 0.1 \times 0.48 = 1.0 - 0.048 = 0.952$$

$$b: 0 - 0.1 \times 0.80 = 0 - 0.080 = -0.080$$

पायरी 5: तपासा: चूक कमी झाली का? नव्या weights ने पुन्हा अंदाज:

$$p = 0.936 \times 0.8 + 0.952 \times 0.6 + (-0.08)$$

$$= 0.7488 + 0.5712 - 0.08$$

$$= 1.24$$

$$\text{नवी loss} = (1.24 - 1)^2 = 0.24^2 = 0.0576$$

बघा: चूक 0.16 वरून 0.0576 वर घसरली! एका पावलात. थेंबू 1.4 वरून 1.24 कडे, म्हणजे खऱ्या 1 च्या **जवळ** आला. हे पुन्हा-पुन्हा (पायरी 1-5) केलं तर थेंबू हळूहळू 1 वर स्थिरावेल: **तो शिकला**. आणि हेच, नेमकं हेच, ChatGPT ट्रिलियन वेळा करतं: फक्त 3 weights ऐवजी अब्जावधी, आणि आर्द्रता-ढग ऐवजी शब्दांचे बाण. **तुम्ही आत्ता, हाताने, यंत्र-शिक्षण केलंत.**

इथे लोक काय चुकीचं समजतात

“यंत्र-शिक्षण म्हणजे काहीतरी गूढ, अतिमानवी.” तुम्ही ते आत्ता कागदावर केलंत: गुणा, बेरीज, वजा. पाच पायऱ्या. गूढ काहीच नाही: फक्त अब्जावधी weights आणि ट्रिलियन पावलं, म्हणून GPU आणि महिने (Book 4, 1.1). “यंत्र जादूने शिकतं” ही कविता संपली; “forward-loss-backward-update, पुन्हा” हे अंकगणित उरलं. आणि हीच सर्वात मोठी मुक्ती: **AI मध्ये कुठेही जादू नाही, फक्त आकडे: आणि आकडे तुम्हाला समजतात.**

नकाशावर

3.1-3.3 derivative | gradient | chain rule
 3.4 थेंबू हाताने: forward (अंदाज 1.4) -> loss (0.16) ->
 backward (उतार गुणा) -> update (weights कमी) -> नवी
 loss 0.0576; हेच ChatGPT, फक्त अब्जावधी पट

सूत्रावर: हा chapter पूर्ण सूत्राचं मूर्त रूप आहे: चूक **मोजली** (0.16), उतार **मोजला**, म्हणून **सुधारता आली** (0.0576). जे मोजता येतं ते सुधारता येतं: यंत्र-शिक्षण म्हणजे या एका वाक्याची ट्रिलियन-पट पुनरावृत्ती.

स्वतः बघा (5 मिनिटं)

वरचं पूर्ण उदाहरण स्वतः कागदावर, हाताने, पुन्हा करा: पण वेगळ्या सुरुवातीच्या weights ने (उदा $w_1 = 0.5$, $w_2 = 0.5$, $b = 0$). पाच पायऱ्या चालवा आणि बघा: loss कमी होते का? (होईल.) एकदा हाताने केलंत की “training” हा शब्द तुमच्यासाठी कायमचा अंकगणित होईल, जादू नाही: आणि तीच या पुस्तकाची सर्वात मोठी कमाई.

विचार करा

थेंबू शिकला. पण एक गोष्ट खटकली: “मी पाऊल-आकार 0.1 घेतला. जर 0.01 (खूप छोटं) घेतलं असतं तर? किंवा 2 (खूप मोठं)? हे ‘0.1’ कुठून आलं, आणि ते चुकीचं निवडलं तर काय होतं? आणि धुक्यातल्या डोंगरावर मी ‘खऱ्या’ तळाऐवजी एका छोट्या खड्ड्यात अडकलो तर?”

पाऊल-आकार (learning rate) हा यंत्र-शिक्षणातला सर्वात नाजूक निर्णय आहे: खूप छोटं = कासवगती; खूप मोठं = तळ ओलांडून उडी. आणि खड्ड्यात-अडकणं हे खरं दुखणं. दोन्ही: पुढचा chapter.

Chapter 3.5 [SPINE]: पाऊल-आकार आणि अडकणं

थेंबूचे दोन प्रश्न: पाऊल-आकार 0.1 कुठून, आणि खड्ड्यात अडकलं तर? हे यंत्र-शिक्षणातले सर्वात व्यावहारिक, सर्वात नाजूक मुद्दे आहेत.

पाऊल-आकार (learning rate): सर्वात नाजूक कळ. gradient descent मध्ये उतार-दिशा ठरली (3.2); पण किती मोठं पाऊल टाकायचं ते learning rate ठरवते (थेंबूचा 0.1). धुक्यातल्या डोंगराची उपमा:

खूप छोटं (0.001): चिमुकली पावलं; तळ गाठायला अनंत काळ (कासवगती). training महिने-वर्ष रेंगाळतं, पैसा जळतो.
 खूप मोठं (2.0): अवाढव्य उड्या; तळ ओलांडून पलीकडच्या चढावर, मग परत ओलांडून: उड्या मारत भरकटतं, कधीच स्थिरावत नाही (किंवा loss स्फोटक वाढते).
 योग्य (0.1 आसपास): नीट उतरत, वाजवी वेगात तळाकडे.

आणि गंमत: **योग्य learning rate कामागणिक, यंत्रागणिक वेगळा**; तो शोधायला प्रयोग लागतात (म्हणून तो “hyperparameter”: यंत्र स्वतः शिकत नाही, माणूस/प्रयोग ठरवतो). आधुनिक पद्धती learning rate **चालता-चालता बदलतात** (सुरुवातीला मोठा: वेगात उतरा; शेवटी छोटा: तळाशी नाजूक): याला “schedule” म्हणतात. पण गाभा तोच: **पाऊल खूप छोटं = वाया वेळ; खूप मोठं = भरकटणं; मधलं सोनं**. आयुष्यातही तेच: सुधारणा खूप हळू = कंटाळा; खूप आक्रमक = अस्थिरता.

अडकणं: खऱ्या तळाऐवजी छोटा खड्डा (local minimum). धुक्यात उतरताना तुम्ही एका छोट्या खड्ड्यात याल: सगळीकडे वर दिसतं (उतार शून्य), म्हणून “तळ आला” वाटतं: पण शेजारी अजून खोल दरी असते, दिसत नाही. यंत्रही असंच “पुरेसं-चांगल्या” जागी थांबू शकतं, “सर्वोत्तम” नाही (3.2 चा इशारा). हे टाळायला युक्त्या: (1) **थोडा गोंधळ (randomness)**: प्रत्येक पावलाला पूर्ण data ऐवजी **तुकडा** वापरणं (mini-batch): यामुळे उतारात थोडा “थरथराट” येतो, जो छोट्या खड्ड्यातून बाहेर ढकलतो: गंमत अशी की **थोडा गोंधळ शिकण्याला मदत करतो**. (2) **गती (momentum)**: घरंगळत्या चेंडूला गती असते: तो छोट्या खड्ड्यावरून **पुढे** जातो (जडत्वाने), खऱ्या खोल दरीत जाऊन थांबतो.

आणि इथे आश्चर्याची गोष्ट (पुढच्या chapter चं बीज): अब्जावधी-मितींच्या डोंगरावर, “छोट्या खड्ड्यात अडकणं” ही अपेक्षेपेक्षा **खूप कमी** समस्या ठरते: बहुतेक “सपाट” जागा सगळ्या दिशांनी खड्डे नसतात (काही दिशांनी खाली जाता येतं). का हे नीट कुणालाच माहीत नाही: पण प्रत्यक्षात gradient descent हे “पुरेसं-चांगलं” आश्चर्यकारकपणे विश्वासाने शोधतं. हे “का चालतं” हे गूढ पुढच्या chapter चा विषय.

इथे लोक काय चुकीचं समजतात

”मोठा learning rate = वेगात शिकेल.” उलट अनेकदा: खूप मोठा = तळ ओलांडून भरकटणं, loss वाढणं, training कोसळणं. वेग learning rate वाढवून येत नाही, योग्य learning rate ने येतो. आणि दुसरा गैरसमज: “randomness म्हणजे चूक/गोंधळ, तो टाळावा.” इथे उलट: थोडा नियंत्रित गोंधळ (mini-batch, momentum) यंत्राला छोट्या खड्ड्यांतून सोडवतो: **कधीकधी थोडी अनिश्चितता हीच सुटकेची वाट.** (आयुष्यातही: सगळं-ठरलेलं मार्ग कधी स्थानिक- खड्ड्यात अडकवतो; थोडा प्रयोग बाहेर काढतो.)

नकाशावर

3.1-3.4 derivative | gradient | chain rule | थेंबू हाताने
3.5 learning rate = पावलाचा आकार (छोटं=कासव, मोठं=भरकटणं, मधलं सोनं; hyperparameter); अडकणं (local minimum) -> थोडा गोंधळ (mini-batch) + गती (momentum) सोडवतात

सूत्रावर: learning rate हा चष्मा दाखवतो की सुधारणेचा वेग हाही एक नाजूक सौदा आहे (Book 5 चा आत्मा इथे!): खूप हळू वाया, खूप वेगात अस्थिर. आणि “मोजून सुधारणं” ला योग्य पाऊल- आकार लागतो: नाहीतर मोजमाप असूनही सुधारणा फसते.

स्वतः बघा (5 मिनिटं)

कुठलीही सवय सुधारण्याचा विचार करा (व्यायाम, अभ्यास). खूप छोटं पाऊल (रोज 1 मिनिट) = प्रगती नगण्य, कंटाळा. खूप मोठं (रोज 5 तास एकदम) = दुसऱ्या दिवशी सोडून देणं (भरकटणं). योग्य पाऊल (रोज 30 मिनिटं, वाढवत) = टिकाऊ प्रगती. Learning rate चं तत्त्व तुमच्या स्वतःच्या सुधारणेलाही लागू: आणि 90-दिवस- roadmap (The Climb) चा वेगही याच तर्काने ठरवा.

विचार करा

थेंबूला सगळं यंत्र कळलं: अंदाज, चूक, उतार, पाऊल. पण एक खोल अस्वस्थता उरली: “हे सगळं चालतं, मी हाताने बघितलं. पण का चालतं? अब्जावधी weights, धुक्यातला अवाढव्य डोंगर, आणि तरी यंत्र काहीतरी उपयोगी शिकतं: नुसतं आकडे जुळवत, कुणी ‘अर्थ’ न शिकवता. हे इतकं साधं गणित इतकं गुंतागुंतीचं वर्तन कसं घडवतं? कुणाला तरी नक्की माहीत असेल ना?”

प्रामाणिक उत्तर तुम्हाला हादरवेल: **नीट कुणालाच माहीत नाही.** जगातले सर्वात मोठे मेंदू अजून “हे का इतकं चांगलं चालतं” यावर भांडतात. आणि तेच न-कळणं हा विभागाचा सर्वात प्रामाणिक, सर्वात मुक्त करणारा धडा आहे. शेवटचा chapter.

Chapter 3.6 [DEPTH]: हे का चालतं: प्रामाणिक गूढ [दोन्ही बाजू]

थेंबूचा प्रश्न: हे साधं गणित इतकं गुंतागुंतीचं वर्तन का घडवतं? आणि उत्तर तुम्हाला हादरवेल आणि मुक्तही करेल: **नीट कुणालाच माहीत नाही**. हा chapter त्या प्रामाणिक गूढाचा, आणि तो पूर्ण पुस्तकातला सर्वात मोकळा श्वास आहे. [DEPTH]: थोडं तत्त्वज्ञान, पण महत्त्वाचं.

गूढ काय आहे. आपण शिकलो: gradient descent धुक्यात पावलापावलाने तळ शोधतं. पण गणिती तर्काने अनेक गोष्टी **चालायला नकोत**:

- अब्जावधी-मितींचा डोंगर म्हणजे अगणित खड्डे: यंत्र एखाद्या वाईट खड्ड्यात अडकायला हवं (3.5). पण अडकत नाही. का?
- यंत्राकडे इतके weights की ते training-data नुसतं **घोकून** (पाठ करून) टाकायला हवं, नव्या गोष्टींवर फसायला हवं (overfitting). पण अनेकदा ते नव्यावरही चांगलं चालतं (generalization). का?
- "मोठं यंत्र = जास्त घोकंपट्टी = वाईट" ही जुनी सांख्यिकी-अपेक्षा; प्रत्यक्षात मोठं यंत्र **जास्त चांगलं** चालतं (Book 4 चे scaling laws). हे जुन्या सिद्धांताच्या **उलट**.

म्हणजे AI **प्रत्यक्षात चालतं**, पण **का इतकं चांगलं चालतं** याचा पूर्ण, समाधानकारक सिद्धांत जगाकडे नाही. Deep learning हे विज्ञानापेक्षा **किमया** (alchemy) जास्त आहे: प्रयोगाने काय चालतं ते माहीत, पण का ते अर्धवट.

दोन बाजू (हा जिवंत वाद):

"सिद्धांत मागे आहे" बाजू: आपल्याकडे भक्कम गणिती स्पष्टीकरण नाही; बरंचसं "हे करून बघितलं, चाललं" (प्रयोग > सिद्धांत). काही संशोधक याला अस्वस्थ होतात: "आपण न-समजलेलं यंत्र जगात सोडतोय."

"अंशतः समजतंय" बाजू: काही तुकडे कळलेत (उच्च-मितीत खड्डे कमी अडकवणारे; थोडा गोंधळ (3.5) मदत करतो; साधेपणाकडे झुकाव). पूर्ण चित्र नाही, पण धुकं विरतंय.

निकाल: AI हे "बांधता येतं पण पूर्ण समजत नाही" या विचित्र जागी आहे: आणि तेच त्याला रोमांचक आणि थोडं भीतीदायक करतं.

हे तुमच्यासाठी मुक्त करणारं का (सर्वात मोठा धडा): जर जगातल्या सर्वात मोठ्या मेंदूंनाही “हे का चालतं” पूर्ण माहीत नसेल, तर तुम्हाला “सगळं समजायलाच हवं” हा दबाव नाही. या क्षेत्रात कुणीच “पूर्ण जाणकार” नाही; सगळे शिकत, प्रयोग करत, आश्चर्य करत पुढे जातायत. म्हणजे तुम्ही “उशिरा आलात, सगळं आधीच माहीत आहे” ही भीती खोटी: सीमारेषेवर सगळेच नवखे आहेत. आणि दुसरा धडा (Book 4 शी पूल): जिथे सिद्धांत अपुरा, तिथे प्रयोग आणि मोजमाप (evals!) हेच सत्याचं साधन: म्हणून जो नीट मोजतो-तपासतो तो, सिद्धांत नसतानाही, पुढे जातो.

आणि सूत्राचं सर्वात खोल रूप: आपण “मोजता येतं ते सुधारता येतं” म्हणालो. Deep learning हे त्याचं टोक आहे: आपल्याला “का” पूर्ण कळत नाही, पण “किती चूक” मोजता येते, म्हणून सुधारता येते. समजून नाही, मोजून प्रगती. हे विचित्र वाटतं, पण मानवजातीने अनेक गोष्टी आधी वापरल्या, मग समजल्या (आग, औषध, वीज). AI तिथेच आहे: वापरात पुढे, समजुतीत मागे. आणि त्या दरीत उभं राहायला भीती नको, नम्रता हवी: तीच खरी वैज्ञानिक वृत्ती.

इथे लोक काय चुकीचं समजतात

“तज्ज्ञांना AI पूर्ण समजतं, फक्त मला नाही.” खोटं, आणि म्हणून घातक (न्यूनगंड देणारं). प्रत्यक्षात frontier वर सगळे अंधारात चाचपडतायत: papers “आम्हाला आश्चर्य वाटलं की...” ने भरलेली असतात. आणि उलट गैरसमज: “समजत नाही म्हणजे सगळं थोटांड.” तेही खोटं: चालतं हे मोजता येतं (benchmarks, प्रत्यक्ष वापर). खरं मधे: चालतं (सिद्ध), पण का पूर्ण नाही (प्रामाणिक). ही नम्र, अचूक भूमिका: आणि तीच तुम्हाला भुलवणाऱ्या दोन्ही टोकांपासून (अति-भक्ती, अति-तुच्छता) वाचवते.

नकाशावर

3.1-3.5 derivative...learning rate

3.6 “का चालतं”: पूर्ण सिद्धांत नाही (किमया > विज्ञान);
सीमेवर सगळे नवखे (मुक्त करणारं); समजून नाही, मोजून
प्रगती; नम्रता = खरी वृत्ती

सूत्रावर: इथे सूत्र त्याच्या सर्वात धाडसी रूपात: पूर्ण न समजताही, मोजता आलं तर सुधारता येतं. Deep learning हे याचा जिवंत पुरावा: आणि म्हणूनच मोजण्याची (evals, गणित) कला समजण्याच्या कलेइतकीच, कधी जास्त, मौल्यवान.

स्वतः बघा (5 मिनिटं)

एखादी गोष्ट आठवा जी तुम्ही **वापरता** पण पूर्ण **समजत** नाही (वीज कशी वाहते, औषध कसं काम करतं, पचन). तरी तुम्ही ती वापरता, आणि “किती चाललं” मोजता (दिवा लागला, ताप उतरला). AI तिथेच आहे. “पूर्ण समजल्याशिवाय वापरू नये” हा दबाव झुगारा: मानवजात नेहमी वापर-आणि-समज यांच्या दरीत जगत आलीये, आणि पुढे गेलीये: मोजत, तपासत, नम्रपणे.

विचार करा

विभाग 3 संपला: उताराचं जग, यंत्र-शिक्षणाचं हृदय, हातात. थेंबूला अंदाज, चूक, उतार, शिकणं, आणि “का चालतं” चं गूढही कळलं. पण एक शेवटचा प्रवास उरला: “मी थेंबूवर सगळं शिकलो: 2 आकडे, 3 weights. पण खरं ChatGPT चं हृदय **attention** आहे (Book 2). ते नक्की या गणितात कसं बसतं? माझे dot products, matrices, softmax: ते सगळं attention मध्ये एकत्र येतं का? मला ते **आकड्यांनी**, पायरी-पायरीने बघायचंय.”

हाच शेवटच्या विभागाचा कळस: थेंबूचं सगळं गणित घेऊन **attention आणि transformer आकड्यांनी, पायरी-पायरीने** उलगडणं. Part 1-2-3 सगळं एकत्र येऊन ChatGPT चं हृदय उघडं होईल. परत AI कडे, Part 4.

BOOK 7, PART 3 चा शेवट

जे तुमच्याकडे आता आहे

derivative	बदलाचा दर ("इथे हललं तर तिथे किती"); पायाखालचा उतार; loss कमी करायला उलट दिशेला जा
gradient	सगळ्या weights चे उतार एकत्र (बाण); सर्वात तीव्र चढ -> उलट जा (gradient descent); धुक्यात पावलापावलाने
chain rule	प्रत्येक थराचा दर गुणत loss मागे वाटणं (backprop); forward (अंदाज) + backward (दोष-वाटप) = शिकणं
थेंबू हाताने	forward 1.4 -> loss 0.16 -> उतार गुणा -> weights कमी -> नवी loss 0.0576; हेच ChatGPT, अब्जावधी पट
learning rate	पावलाचा आकार (छोटं=कासव, मोठं=भरकटणं); अडकणं -> थोडा गोंधळ + गती सोडवतात
का चालतं	पूर्ण सिद्धांत नाही (किमया); सीमेवर सगळे नवखे; समजून नाही, मोजून प्रगती; नम्रता = वृत्ती

स्वतःची परीक्षा (4 प्रश्न, बोलून; उत्तरं खाली)

1. "यंत्र शिकतं" चं पूर्ण गणित पाच पायऱ्यांत?

forward (अंदाज) -> loss (चूक मोजा) -> backward (chain rule ने प्रत्येक weight चा उतार) -> update (उताराच्या उलट, learning rate ने) -> पुन्हा. loss घसरते. (3.4)

2. Gradient बाण काय दाखवतो, आणि आपण काय करतो?

सर्वात तीव्र **चढाची** दिशा; loss कमी करायला त्याच्या **उलट** पाऊल (gradient descent). (3.2)

3. Backpropagation म्हणजे "यंत्र मागे विचार करतं" का?

नाही: शुद्ध गुणाकार: प्रत्येक थराचा दर गुणत loss मागे, प्रत्येक weight ला त्याचा दोष-वाटा. बुद्धी नाही, अंकगणित. (3.3)

4. "AI का चालतं" चं प्रामाणिक उत्तर, आणि त्याचा धडा?

पूर्ण कुणालाच माहीत नाही (किमया > विज्ञान); म्हणून सीमेवर सगळे नवखे (मुक्त करणारं), आणि जिथे सिद्धांत अपुरा तिथे मोजमाप (evals) हेच सत्य. (3.6)

पुढच्या भागाची चाहल

थेंबूचं सगळं गणित (बाण, dot, matrix, probability, softmax, gradient) आता attention मध्ये एकत्र येईल. Part 4: परत AI कडे: attention आकड्यांनी पायरी-पायरी, एक transformer-मजला हाताने, scaling laws चं गणित, आणि एक खरा paper उलगडून वाचणं. ChatGPT चं हृदय, पूर्ण उघडं.

BOOK 7, PART 3 चा MINI-GLOSSARY

backpropagation	chain rule ने loss मागे वाटणं (3.3)
chain rule	साखळीतल्या दरांचा गुणाकार (3.3)
derivative	बदलाचा दर; एका दिशेतला उतार (3.1)
gradient	सगळ्या weights च्या उतारांचा बाण (3.2)
gradient descent	उताराच्या उलट पावलं टाकत तळ शोधणं (3.2)
learning rate	पावलाचा आकार (hyperparameter) (3.5)
local minimum	खरा तळ नसलेला छोटा खड्डा (3.5)
mini-batch	data चा तुकडा वापरून पाऊल; थोडा गोंधळ (3.5)
momentum	घरंगळत्या चेंडूची गती; खड्ड्यातून सोडवते (3.5)
overfitting	training घोकणं, नव्यावर फसणं (3.6)